MacReduce: Federated QLoRA on GPU and Neural Engine Clients

Gautam Diwan
Carnegie Mellon University
gdiwan

Abstract

We fine-tune a language model by treating the processors in a Mac as federated clients, then repeating that grouping across Macs. A client is a GPU, a GPU bound together with the Neural Engine, or those two processors kept separate. Every client trains on a shard of one WikiText-2 mixture, so the heterogeneity is the device. The quantized base stays on the client. The network carries a LoRA adapter delta. Validation perplexity is the quality metric, and adapter bytes are the communication metric. The baseline is FedAvg on one GPU.

1 Introduction

The Apple Neural Engine is a 16-core matrix processor next to the CPU and the GPU. MLX fine-tuning never schedules it, so that unit sits idle while the GPU trains. Apple exposes it through Core ML, for inference, and Core ML may still run the graph on the GPU. ANEForge reverse-engineered the private `aned` path and compiles a Python graph, forward and backward, onto the engine. Orion did the same in Objective-C for a small transformer: attention and the feed-forward block run on the engine, and Adam runs on the CPU. That is what makes an engine client possible.

FedAvg takes local steps before an average [1]. Unequal step counts use the FedNova rescaling, labeled as plain SGD when the solver is AdamW [2]. Int8 codes keep an error residual. Star, ring, and tree agree before quantization.

2 Method

We will train three cohorts, and use the same cohort on every Mac. The first is n GPUs. The second is n combined clients, each a GPU and a Neural Engine that send one adapter delta. The third is n GPUs and n Neural Engines, as separate clients. Every client trains on a shard of the same WikiText-2 mixture. The base stays on the client. The network carries only the LoRA delta. Chaining more than two Macs is an open part of this design, and we invite teammates to take that measurement.

Each round starts from the broadcast adapter. FedAvg averages after a block of local AdamW steps. The control averages after every step. The average is a star, a ring, or a tree, with the FedNova rescaling when step counts differ. We will also send the delta as int8 codes plus an error residual, and count the base-weight bytes that stayed home. We will run these exchanges over the network. A wired link, including Thunderbolt, is a follow-up we may try once the network numbers are in.

The model is QLoRA rank 8 on `mlx-community/Qwen3-0.6B-4bit`. The GPU client uses MLX. The engine client uses ANEForge or Orion. Single-machine numbers come from an M4 MacBook Air with 10 CPU cores (4 performance, 6 efficiency), 10 GPU cores, a 16-core Neural Engine, and 24 GB of unified memory.

3 Evaluation

The dataset is Salesforce WikiText-2 raw v1 at revision `b08601e04326c79dfdd32d625aee71d232d685c3`. Training uses the first 4,096 lines for three epochs, batch size 8, and two gradient-accumulation steps. Validation and test stay the full splits. The metric is validation perplexity at each epoch, plus adapter bytes per round. The baseline is one Mac, GPU only, FedAvg, on a star, with an uncompressed delta. The same metric is reported for n GPUs, for n combined GPU and Neural Engine clients, and for n GPUs together with n Neural Engines. A Neural Engine point counts when a compute plan or a power trace shows the engine ran.

References

[1] B. McMahan et al. Communication-efficient learning of deep networks from decentralized data. *AISTATS*, 2017.

[2] J. Wang et al. Tackling the objective inconsistency problem in heterogeneous federated optimization. *NeurIPS*, 2020.